

Algorithms for Data Science

Exercise 2

(with Solutions)

See the cheatsheet for valuable remarks!

In our course: $\log$ always means $\log_2$.

Task 2.1: Oh-Notation

- (a) Someone tells you that an algorithm A has running time $O(n \log n)$ and an algorithm B has running time $O(n^2)$. Do you know which algorithm is faster?

Solution: The answer is no. You don't know it. You only know that the running times of the two algorithms are asymptotically not higher than what is given in the O -notations. It is exactly the same situation if someone told you that one building a is not higher than 10 meters, whereas another building b is not higher than 15 meters. You cannot tell which building is higher or if they are of equal height.

If the running times of A and B had been given in Θ notation, then we could compare them and conclude, that A with $\Theta(n \log n)$ is asymptotically faster than B with $\Theta(n^2)$. (Why? By the “LOG < POL” Rule from the Cheatsheet, $\log n = O(n)$ and, thus, $n \cdot \log n = O(n \cdot n) = O(n^2)$.) Note that we could also come to the same conclusion if the running time of A had been given as $\Omega(n \log n)$ with the running time of B given as $O(n^2)$.

Simplify the following running times. Explain the simplifications.

Difficult sample tasks explained by the tutor.

(b) $\Theta(10 \cdot n^2 + 978 \cdot n + n \cdot n \cdot 2 \cdot n/1000)$

Solution: $\Theta(n^3)$.

Formal proof: First observe that $n \cdot n \cdot 2 \cdot n/1000 = n^3/500$. Next, separately show that $10n^2 + 978 \cdot n + n^3/500$ is $O(n^3)$ and $\Omega(n^3)$.

- $\Omega(n^3)$ as $10n^2 + 978 \cdot n + n^3/500 \geq n^3/500$, and, therefore, by picking the constants $C = 1/500$ (or any smaller value) and $n_0 = 0$, we obtain, for all $n > n_0$ that $10n^2 + 978 \cdot n + n^3/500 \geq C \cdot n^3$.
- $O(n^3)$ as

$$\begin{aligned} 10n^2 + 978 \cdot n + n^3/500 &\leq 10n^3 + 978 \cdot n^3 + n^3/500 \\ &= (10 + 978 + 1/500)n^3 \\ &\leq 979.002 \cdot n^3, \end{aligned}$$

and, therefore, by picking the constants $C = 979.002$ (or any larger value, e.g., 1000) and $n_0 = 0$, we obtain, for all $n > n_0$ that $10n^2 + 978 \cdot n + n^3/500 \leq C \cdot n^3$.

Informal proof (using the simplification rule from the cheatsheet):

1. Remove constant factors: $\Theta(n^2 + n + n \cdot n \cdot n)$.
2. Transform complicated terms: $\Theta(n^2 + n + n^3)$.
3. Choose the fastest growing term: n^3 .

(c) $\Theta(1000(\log_5 n)^{1000} + \sqrt{n} \cdot 2000 + (11/10)^n)$

Solution: $\Theta(1.1^n)$

Informal proof:

1. Remove constant factors: $\Theta((\log_5 n)^{1000} + \sqrt{n} + (11/10)^n)$.
2. Transform complicated terms: $\Theta((\log_5 n)^{1000} + \sqrt{n} + 1.1^n)$.
3. Choose the term that grows the fastest: 1.1^n .

Explanation: By the “LOG < POL” rule (see the cheatsheet), $(\log_5 n)^{1000}$ grows slower than the polynomial $\sqrt{n} = n^{1/2}$, so we can remove the first term. By the “POL < EXP” rule, $\sqrt{n}$ grows slower than the exponential 1.1^n , so we remove also $\sqrt{n}$, which leaves us with only one term: 1.1^n .

(d) $\Theta(4 \cdot n^{1/4} \cdot n^{0.25} + 2(2^n)^{2/n} + 50 \log_2(100n) + \log_3(9^n))$

Solution: $\Theta(n)$

Informal proof:

1. Remove constant factors: $\Theta(n^{1/4} \cdot n^{0.25} + (2^n)^{2/n} + \log_2(100n) + \log_3(9^n))$.

2. Transform complicated terms: $\Theta(n^{1/2} + 1 + \log_2 n + n)$
 Explanation: Apply the transformation rules of the cheatsheet.

- $n^{1/4} \cdot n^{0.25} = n^{1/4} \cdot n^{1/4} = n^{1/4+1/4} = n^{2/4} = n^{1/2}$.
- $(2^n)^{2/n} = 2^{n \cdot 2/n} = 2^2 = 4$.
- $\log_2(100n) = \log_2 100 + \log_2 n$.
- $\log_3(9^n) = n \log_3 9 = n \cdot 2$.

Thus, we obtain $\Theta(n^{1/2} + (4 + \log_2 100) + \log_2 n + 2n)$. By further simplifying constant factors, we get $\Theta(n^{1/2} + 1 + \log_2 n + n)$.

3. Choose the term that grows the fastest: n

Explanation:

By the “LOG < POL” and “POL < EXP” rule, the fastest growing term is n .

Regular tasks for the students.

- (e) $\Theta(91n + n^2 \cdot 4n^2 + 3 \log n + 42n^3)$

Solution: $\Theta(n^4)$.

Informal proof:

1. Remove constant factors: $\Theta(n + n^2 \cdot n^2 + \log n + n^3)$
2. Transform complicated terms: $\Theta(n + n^4 + \log n + n^3)$
3. Choose the fastest growing term: n^4 .

Formal proof: We have to separately show that $91n + 4n^4 + 3 \log n + 42n^3$ is $O(n^4)$ and $\Omega(n^4)$.

- $\Omega(n^4)$ as $91n + 4n^4 + 3 \log n + 42n^3 \geq n^4 = C \cdot n^4$ for $C = 1$ and for all $n \geq n_0 = 1$.
- $O(n^4)$ as

$$\begin{aligned} 91n + 4n^4 + 3 \log n + 42n^3 &\leq 91n^4 + 4n^4 + 3n^4 + 42n^4 \\ &= (91 + 4 + 3 + 42)n^4 \\ &= C \cdot n^4 \end{aligned}$$

for $C = 91 + 4 + 3 + 42 = 139$ and all $n \geq n_0 = 1$.

- (f) $\Theta(1000^{1000^{1000}} + 1/2 \cdot 1.0001^n + 40 \cdot n^{100})$

Solution: $\Theta(1.0001^n)$

Informal proof:

1. Remove constant factors (or replace with 1): $\Theta(1 + 1.0001^n + n^{100})$.
2. Transform complicated terms: *No need, the terms are already simple.*
3. Choose the fastest growing term: 1.0001^n .

Explanation: Obviously, it is not 1. By the “POL < EXP” rule, 1.0001^n is growing faster than n^{100} .

(g) $\Theta(5 \log(5n))$.

Solution: $\Theta(\log n)$ using $\log(5n) \geq \log n$ and $\log(5n) \leq \log(n^2) = 2 \log n$ (for $n \geq 5$). (We applied a transformation rule from the cheatsheet.)

(h) $\Theta(5 \cdot 2^{5n+5})$.

Solution: $\Theta(2^{5n})$ (or $\Theta(32^n)$) using that $2^{5n+5} = 2^5 \cdot 2^{5n} = \Theta(2^{5n})$. We cannot simplify $\Theta(2^{5n})$ further to $\Theta(2^n)$ as $2^{5n} = O(2^n)$ would imply that there are constants $c > 0$ and $n_0 \geq 0$ such that, for all $n \geq n_0$, $2^{5n} \leq c \cdot 2^n \Leftrightarrow 2^{4n} \leq c$; a contradiction. Another way to see it: $2^{5n} = (2^5)^n = 32^n$, which grows much faster than 2^n .

(i) $\Theta(42m + k/100 + m \cdot 21 \cdot k + 2^{2 \log_2 n})$. *Sometimes there is more than one parameter.*

Solution: $\Theta(mk + n)$

Informal proof:

1. Remove constant factors: $\Theta(m + k + mk + 2^{2 \log_2 n})$.
2. Transform complicated terms: $\Theta(m + k + mk + n^2)$.
Explanation: by the transformation rules from the cheatsheet, we have $2^{2 \log_2 n} = (2^{\log_2 n})^2$, which, by the definition of the logarithm, is equal to $(n)^2 = n^2$.
3. Choose the fastest growing terms (not growing slower than another term): mk and n^2 .
Explanation: m and k grow slower than mk . Both mk and n^2 do not grow slower than any other term, especially, they are incomparable to each other, so we take both. (We would need additional knowledge on the relationship of n , k and m in order to be able to simplify the running time further.)

Bonus Tasks

Tasks for solving at home (not necessarily discussed in classes); relevant for short tests and the exam.

Task 2.2: Oh-Notation++

(a) You are given the following running time functions (in Θ -notation). Order them by order of growth!

$n!$ $(\log n)^2$ n^2 $\sqrt{n}$ $2^{\sqrt{2} \log n}$ 2^{2n} $n \cdot 2^n$ $\log(n^2)$ $4^{\log n}$

Solution:

1. $\log(n^2) = 2 \log n$
2. $(\log n)^2$
3. $\sqrt{n}$
4. $2^{\sqrt{2} \log n} = n^{1.5 \dots}$
5. n^2 and $4^{\log n} = 2^{2 \log n} = n^2$

6. $n \cdot 2^n$

7. $2^{2n} = 2^n \cdot 2^n$ (and therefore larger than the function $n \cdot 2^n$ above)

8. $n! \approx n^n$

*For our course, it suffices to remember that $n!$ grows almost as fast as n^n which is much faster than any exponential function. The following discussion offers optional enrichment for the curious student and is **not mandatory** for our course:*

To be precise, $n!$ grows slower than n^n ($n! \neq \Omega(n^n)$), however, people have shown that $n! \geq n^{0.9999n}$ for sufficiently large n .

For our task, we just have to show that $n!$ grows faster than the exponential function 2^{2n} . We can do it as follows:

By definition, $n! = 1 \cdot 2 \cdot \dots \cdot (n-1) \cdot n$. Thus,

$$\begin{aligned} n! &> n/2 \cdot (n/2 + 1) \cdot \dots \cdot (n-1) \cdot n \\ &> (n/2) \cdot \dots \cdot (n/2) \\ &= (n/2)^{n/2} \\ &= (n/2)^{2n/4} . \end{aligned}$$

If we assume that $n \geq 32$, we can replace the n in the base by 32, that is, we can write

$$(n/2)^{2n/4} \geq (32/2)^{2n/4} = 16^{2n/4} = (16^{1/4})^{2n} = 2^{2n} .$$

(We used the transformation rule and the fact that $2^4 = 16$, that is, $16^{1/4} = 2$.) Hence, $n! > 2^{2n}$ for all $n \geq 32$.

Prove your answers for the following questions.

(b) Is $f(n) + g(n) = \Theta(\max(f(n), g(n)))$? (Assume that f and g are nonnegative functions.)

Solution: Yes!

First observe that $f(n) + g(n) \geq \max(f(n), g(n)) = \Omega(\max(f(n), g(n)))$.

Next, observe that, $f(n) + g(n) \leq 2 \max(f(n), g(n)) = O(\max(f(n), g(n)))$.

In total, Θ holds as O and Ω hold.

(c) Is $(n + 100)^5 = \Theta(n^5)$?

Solution: Yes!

First observe that $(n + 100)^5 \geq n^5 = \Omega(n^5)$.

Next, observe that, $(n + 100)^5 \leq (n \cdot 100)^5 = n^5 \cdot 100^5 = O(n^5)$ as 100^5 is a constant.

In total, Θ holds as O and Ω hold.

(d) Is $\log_3 n + \log_5 n = \Theta(\log_7 n)$?

Solution: Yes!

By the transformation rules from the cheatsheet, $\log_3 n = \Theta(\log_7 n)$ and $\log_5 n = \Theta(\log_7 n)$. Thus, $\log_3 n + \log_5 n = \Theta(\log_7 n + \log_7 n) = \Theta(\log_7 n)$,

If you are a curious student, try to prove $\log_3 n = \Theta(\log_7 n)$ yourself using the other transformation rules! (The proof is beyond the scope of the course.)

Continue to read only, if you have tried to prove it yourself!

$$\begin{aligned}\log_3 n &= \log_3(7^{\log_7 n}) \quad (\text{By the definition of the logarithm, } n = 7^{\log_7 n}) \\ &= \log_7 n \cdot \log_3 7 \quad (\text{Transformation rule}) \\ &= \Theta(\log_7 n) \quad (\text{Removing the constant factor } \log_3 7.)\end{aligned}$$

(e) Is $300 \log(n^n k) + k^k + 2^{\sqrt{k} \log k} = O(n \log n + 2^{k \log k})$?

Solution: Yes!

We use the following rules:

- $\log(ab) = \log a + \log b$
- $\log(a^b) = b \log a$
- $a = 2^{\log a}$ and in particular $a^b = 2^{b \log a}$

Before continuing with reading, try to solve it yourself using the rules above!

Using the rules above and the fact that k^k grows faster than $k^{\sqrt{k}}$ and than $\log k$, we obtain

$$\begin{aligned}300 \log(n^n k) + k^k + 2^{\sqrt{k} \log k} &= 300(\log(n^n) + \log k) + k^k + k^{\sqrt{k}} \\ &= 300n \log n + 300 \log k + k^k + k^{\sqrt{k}} \\ &\leq 300n \log n + 300k^k + k^k + k^k \\ &\leq 303(n \log n + k^k) \\ &= 303(n \log n + 2^{k \log k}) \\ &= O(n \log n + 2^{k \log k})\end{aligned}$$

Task 2.3: Oh-Notation Definitions

(a) In the lecture, we have seen running times depending on two parameters. For instance $O(kn)$. Propose a formal definition for $f(n_1, \dots, n_i) = O(g(n_1, \dots, n_i))$.

Solution: $f(n_1, \dots, n_i) = O(g(n_1, \dots, n_i))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that

$$f(n_1, \dots, n_i) \leq c \cdot g(n_1, \dots, n_i)$$

for all $n_1, \dots, n_i$ that satisfy $n_j \geq n_0$ for each $j \in \{1, \dots, i\}$. (Note that n_0 is our fixed constant, whereas $n_1, \dots, n_i$ are the parameters of the functions.)

(b) Prove that $f(n) = \Omega(g(n))$ holds if $g(n) = O(f(n))$ using the definition from the cheatsheet.

Solution: Just look at the definition of O and divide by C . Define $1/C$ as a new constant C' and now observe that the definition of Ω is fulfilled.

Task 2.4: Designing Algorithms: Airfield

Motivation: You want to build an airfield in a mountainous area. Therefore, you look for a longest possible plateau, that is, for a longest possible flat surface. An additional constraint is that the airfield has to be build along a fixed line of latitude since this is the direction in which the airplanes land or take off. To solve this problem, you first measure the height of the surface along the line of latitude at n equidistant sample points.

Now, given an integer array $h[1..n]$ with the measured heights (rounded to full meters), propose an efficient algorithm to find a longest plateau, that is, a longest consecutive subsequence $h[i..j]$ of the array where all the integers have the same value. The output should consist of a position i of the array where a longest plateau starts.

What is the running time of your algorithm?

Solution: $\Theta(n)$ time: Go from left to right and compare the current plateau with the longest found so far.

Algorithm 1: LongestPlateau

Input: array $h[1..n]$ of integers

Output: start position of a longest plateau

$pos = 1$ // The starting position of the longest plateau found so far.

$len = 1$ // The length of the longest plateau found so far.

$cPos = 1$ // The starting position of the current plateau.

$cLen = 1$ // The length of the discovered part of the current plateau.

for $i = 2$ **to** n **do**

if $h[i - 1] == h[i]$ **then**

$cLen++$

if $cLen > len$ **then**

$pos = cPos$

$len = cLen$

else

$cPos = i$

$cLen = 1$

return pos

Task 2.5: Designing Algorithms: Forest Age / Salary Database

- (a) *Motivation 1 (Forest):* Suppose that we have a database d with the age of all n trees of a forest (possibly obtained by some novel age-estimation technique using drones), where $d[i]$ is the age of the i -th tree. We also received m e-mails from different biologists, each one asking how many trees are there of a specific age. We store all the queries in a table q , where, for the j -th e-mail that we received, we store in $q[j]$ the age for which we have to compute the number of trees. Since the biologists are only interested in young trees, we know also that all values in q are smaller than some given small number k . (We assume that all the ages in q and d are rounded to full days.) Your task is to answer all the e-mails!

Motivation 2 (Salary): You maintain a database storing the yearly salary of all n persons in Poland. The database is implemented as an integer array $d[1..n]$ where $d[i]$ is the salary of the i -th person in Polish Zloty (PLN). Now, your boss gives you a small database containing the salary (in PLN) of some other m persons whose salary is not larger than k PLN, also implemented as an integer array $q[1..m]$. Your task is to compute, for each person of the database q , how many persons in the database d have exactly the same salary.

Input: arrays $d[1..n]$ and $q[1..m]$ of nonnegative integer values, and a parameter k with the guarantee that no value in q is larger than k .

Output: an array $res[1..m]$ where $res[i]$ is the number of indices $j \in \{1, \dots, n\}$ for which $q[i] = d[j]$.

Propose an algorithm solving this problem using only $O(1)$ additional memory space, that is, besides d , q , k and res , you are allowed to use only a constant number of variables.

What is the running time of your algorithm?

Solution: Time: $\Theta(n \cdot m)$: For each entry in q , check how many same entries are there in d .

Input: arrays $d[1..n]$ and $q[1..m]$

Output: array $res[1..m]$ where $res[i]$ is the number of times $q[i]$ appears in d .

$res[1..m]$ = new array filled with zeros

for $i = 1$ **to** m **do**

for $j = 1$ **to** n **do**

if $q[i] == d[j]$ **then**

$res[i]++$

return res

- (b) We consider the same problem as in (a). This time, propose an algorithm with running time $O(n + m + k)$ without any constraints regarding the memory space.

What is the extra memory used by your algorithm in Θ notation?

Compare your running time to (a) when assuming $m = \Theta(\sqrt{n})$ and $k = \Theta(m)$.

Hint: Use the counting technique.

Solution: Create an auxiliary array $counts[1..k]$. Go through d : for each i , if $d[i] \leq k$, then increase $counts[d[i]]$ by one. Then, go through q and for each i , look the number up in $counts[q[i]]$.

Input: arrays $d[1..n]$ and $q[1..m]$ where all values in q are bounded by k

Output: array $res[1..m]$ where $res[i]$ is the number of times $q[i]$ appears in d .

$counts[1..k]$ = new array filled with zeros // It takes $\Theta(k)$ time to create and fill this array.

for $j = 1$ **to** n **do**

if $d[j] \leq k$ **then**

 // Otherwise the value $d[j]$ is not interesting for us.

$counts[d[j]]++$

$res[1..m]$ = new array filled with zeros

for $i = 1$ **to** m **do**

$res[i] = counts[q[i]]$

return res

Extra space usage: $\Theta(k)$ for creating the auxiliary array.

Comparison with the first algorithm: If $m = \Theta(\sqrt{n})$ and $k = \Theta(m)$, then the running time of the first algorithm is $\Theta(n\sqrt{n})$ whereas the running time of the second algorithm is $\Theta(n)$.

Task 2.6: *Analyzing Algorithms: SomeSortingAlgo***Algorithm 2:** SomeSortingAlgo

Input: array $a[1..n]$ of integers
Output: array a is sorted non-decreasingly

```

1  $i = 1$ 
2 while  $i \leq n - 1$  do
3    $minPos = i$ 
4    $j = i + 1$ 
5   while  $j < n$  do
6     if  $a[j + 1] < a[minPos]$  then
7        $minPos = j + 1$ 
8      $j++$ 
9   if  $minPos \neq i$  then
10    // exchanges the values at position  $i$  and  $minPos$  in the array  $a$ 
11     $Swap(a, i, minPos)$ 
12   $i++$ 

```

- (a) Write an algorithm for the function $Swap(a, i, \ell)$ that exchanges the values at position i and ℓ in the array a , that is, after calling $Swap(a, i, \ell)$, $a[i]$ contains the former value at $a[\ell]$, and $a[\ell]$ contains the former value at $a[i]$.

Solution:**Algorithm 3:** Swap(a, i, ℓ)

Input: An array $a[1 \dots n]$ and two indices i and ℓ with $1 \leq i, \ell \leq n$.

Output: The array a with the elements at positions i and ℓ swapped.

```

1  $Swap(a, i, \ell)$ 
2    $temp = a[i]$ 
3    $a[i] = a[\ell]$ 
4    $a[\ell] = temp$ 

```

- (b) Give the best case and the worst case running times in Θ -notation of SomeSortingAlgo.

Solution: Best=Worst: $\Theta(\sum_i (n - i)) = \Theta(n^2)$

Explanation:

The first line runs in constant time, but its contribution is negligible due to subsequent code.

In each iteration of the outer loop ($i = 1$ to $n - 1$), the inner loop runs from $j = i + 1$ to $j = n$, resulting in exactly $n - i$ iterations. As each iteration takes constant time, the total time for the inner loop in the i -th outer loop iteration is $\Theta(n - i)$.

The remaining operations in the outer loop run in constant time and are dominated by the inner loop. Summing $\Theta(n - i)$ over all i from 1 to $n - 1$ gives the final running time, as detailed in the formula analysis that follows.

$$\begin{aligned}
&= \Theta(n-1) + \Theta(n-2) + \dots + \Theta(n-(n-1)) &= \sum_{i=1}^{n-1} \Theta(n-i) \\
&= \Theta((n-1) + (n-2) + \dots + (n-(n-1))) &= \Theta\left(\sum_{i=1}^{n-1} (n-i)\right) \\
&= \Theta((n-1) + (n-2) + \dots + 1) \\
&= \Theta(1 + 2 + \dots + (n-1)) &= \Theta\left(\sum_{i=1}^{n-1} i\right) \\
&= \Theta\left(\frac{n(n-1)}{2}\right) & \text{(See cheatsheet)} \\
&= \Theta\left(\frac{n^2}{2} - \frac{n}{2}\right) \\
&= \Theta(n^2)
\end{aligned}$$

- (c) The algorithm has a mistake. Give a shortest possible input array for which the algorithm computes a wrong output. Describe the mistake and fix it by changing the code as little as possible.

Solution: One of many possible answers: If $a = \langle 2, 1 \rangle$, then the output of the algorithm is also $\langle 2, 1 \rangle$ (which is wrong); the correct output would be $\langle 1, 2 \rangle$.

The problem is that the comparison “ $a[j+1] < a[\text{minPos}]$?” within the inner while loop never compares $a[i]$ with $a[i+1]$. Though $\text{minPos} = i$ in the beginning, we always have $j+1 \geq i+2$ due to the mistake that we start with $j = i+1$ in line 4. Consequently, if $a[i+1]$ is the smallest element in $a[i..n]$, we will not notice this fact and we will not move $a[i+1]$ to its correct position i .

To fix the error, it suffices to drop the “+1” in line 4, so that initially we know have $j = i$. The full modified code (changes in green):

Algorithm 4: SomeSortingAlgo

Input: array $a[1..n]$ of integers

Output: array a is sorted non-decreasingly

```

1  $i = 1$ 
2 while  $i \leq n - 1$  do
3    $\text{minPos} = i$ 
4    $j = i$  // Repaired line
5   while  $j < n$  do
6     if  $a[j+1] < a[\text{minPos}]$  then
7        $\text{minPos} = j+1$ 
8      $j++$ 
9   if  $\text{minPos} \neq i$  then
10    // exchanges the values at position  $i$  and  $\text{minPos}$  in the array  $a$ 
11     $\text{Swap}(a, i, \text{minPos})$ 
12   $i++$ 

```

- (d) Prove that in lines 3 to 8, the algorithm that you repaired in task (c) finds a position $minPos$ of a smallest element in $a[i..n]$ (that is, $a[minPos]$ is this smallest element). For your proof, come up with a helpful loop invariant.

Solution:

Invariant (with respect to line 5): “ $a[minPos]$ is a smallest element in $a[i..j]$.”

In the beginning, $minPos = j = i$, thus the subarray $a[i..j] = a[i..i]$ contains only the element $a[i] = a[minPos]$ and the claim of the invariant is trivially true.

Now assume that the claim is true for some j at line 5. If $a[j+1] \not\leq a[minPos]$, then $a[j+1] \geq a[minPos]$ and, obviously, $a[minPos]$ is also a smallest element in $a[i..j+1]$. Since we do not change $minPos$ in this case, the claim remains true for $j+1$. If $a[j+1] < a[minPos]$, then, by our assumption, $a[j+1]$ is smaller than the smallest value in $a[i..j]$ and, hence, $a[j+1]$ is the smallest element in $a[i..j+1]$. Thus, after setting $minPos = j+1$ in line 7, the claim is true for $j+1$.

The above arguments prove that our invariant is correct.

At the end, when we exit the while loop, we have $j = n$. Since our invariant is true, we conclude that $minPos$ is a position of a smallest element in $a[i..j] = a[i..n]$.

- (e) Prove the correctness of the whole algorithm `SomeSortingAlgo` via the following invariant (defined with respect to line 2):

The subarray $a[1..i]$ is sorted non-decreasingly, and there is no element in the subarray $a[1..i-1]$ that is larger than some element in $a[i..n]$. Furthermore $a[1..n]$ contains all the elements that it contained in the beginning. (For $i = 1$, we define $a[1..i-1] = a[1..0]$ as an empty array.)

Recall that you have to prove that the invariant holds (i) in the beginning (when $i = 1$), and (ii) that it holds for each $i > 1$ using the assumption that it holds for $i-1$ (equivalently: that it holds for $i+1$ assuming that it holds for $i \geq 1$). Finally, use the invariant to conclude that at the end the whole input array is sorted.

Solution:

In the beginning (when we reach line 2 for the first time), $i = 1$ and therefore $a[1..i] = a[1]$ which is obviously a sorted array. Since $a[1..i-1] = a[1..0]$ is an empty array, there is indeed no element in $a[1..i-1]$ that is larger than some element in $a[i..n]$. Thus, the invariant holds for $i = 1$.

Now, assume that the invariant holds for some i with $1 \leq i \leq n-1$ at line 2. In lines 3 to 8, the algorithm finds a position $minPos$ of a smallest element in $a[i..n]$ (that is, $a[minPos]$ is this smallest element). We proved this fact in the subtask (d) above. In the next two lines, the algorithm exchanges this smallest element with the element that is on position i (provided that not already $i = minPos$). Thus, in the beginning of line 11, $a[i]$ contains a smallest element from $a[i..n]$, and $a[i..n]$ contains all the elements that it contained before. Since the invariant holds for i , we know that $a[1..i-1]$ is sorted non-decreasingly and that there is no element in $a[1..i-1]$ that is larger than some element in $a[i..n]$. Consequently, since $a[i]$ is a largest element from $a[1..i]$ and a smallest one from $a[i..n]$, we know that $a[1..i+1]$ also sorted non-decreasingly and $a[1..i]$ contains no element larger than some element in $a[i+1..n]$. Thus the invariant holds also for $i+1$ in the beginning of line 11, and, after increasing i by one, the invariant also holds for the next i at line 2.

Now let us use the invariant to show that the algorithm is correct. At the end, when the program leaves the while loop at line 2, the while loop condition has to be violated. Thus, at the end, we have $i > n-1$, which in our algorithm can be only if $i = n$. By putting $i = n$ into the definition of our invariant, we obtain that a contains all the elements as in the beginning and that $a[1..i] = a[1..n] = a$ is sorted non-decreasingly. Hence, our algorithm is correct!

Task 2.7: *Analyzing Algorithms: StrangeAlgo***Algorithm 5:** StrangeAlgo**Input:** nonnegative integers n and k **Output:** ?

```

1 StrangeAlgo( $n, k$ )
2   if  $k == 0$  then
3     return 0
4   if  $k == 1$  then
5     return 1
6    $a[1..1000] = \text{new array}$ 
7    $i = 1$ 
8   while  $i \leq 1000$  do
9      $a[i] = i$ 
10     $i++$ 
11  while  $i \leq n$  do
12     $i++$ 
13   $ret = \text{StrangeAlgo}(n, k - 2)$ 
14  return  $ret$ 

```

- (a) What is the output of
- `StrangeAlgo(10000000000000000, 8941307)`
- ?

Solution: The output is 1 as the algorithm computes $k \bmod 2$, that is, it returns 0 if k is even, and 1 if k is odd. To see this, observe that at each call of `StrangeAlgo`, k is decreased by exactly 2. Thus, at the last call, we have $k = 1$ or $k = 0$, depending whether originally k was odd or even.

- (b) What is the running time of
- `StrangeAlgo( $n, k$ )`
- in
- Θ
- notation in dependence of
- n
- and
- k
- ?

Solution: $\Theta(n \cdot k)$ —`StrangeAlgo` is called $\lceil k/2 \rceil$ times, each time performing $\Theta(n)$ operations. A more detailed proof:
`StrangeAlgo( $n, k$ )` calls `StrangeAlgo( $n, k - 2$ )`, which calls `StrangeAlgo( $n, k - 4$ )`, and so on, until reaching `StrangeAlgo( $n, 1$ )` or `StrangeAlgo( $n, 0$ )`. Thus, `StrangeAlgo` is called $\lceil k/2 \rceil$ times. The final call takes constant time, while each previous call takes $\Theta(n)$ time: the second while loop ($i \leq n$) runs in $\Theta(n)$, and all other lines run in constant time. In particular, the first while loop ($i \leq 1000$) executes at most 1000 times, which is constant. Hence, the total time is $\Theta((\lceil k/2 \rceil - 1)n + 1) = \Theta(n \cdot k)$.

- (c) What is the memory complexity of
- `StrangeAlgo( $n, k$ )`
- in
- Θ
- notation in dependence of
- n
- and
- k
- ?

Solution: $\Theta(k)$ —At each call of `StrangeAlgo`, an array a of constant size 1000 is build. However, this array exists and is not discarded from memory when the next call occurs (it will be discarded only when the function call that has created this array makes its return statement and ends). Thus, at the last call of `StrangeAlgo`—and there are $\lceil k/2 \rceil$ calls—the memory has to maintain $\lceil k/2 \rceil$ such constant arrays and some additional constant many variables (including some auxiliary invisible ones to keep track of the calls) for each call. Thus, the memory consumption in total is $\Theta(k)$.

- (d) Modify `StrangeAlgo(n, k)` to run in time $\Omega(n^k)$.

Solution: A possible solution: We remove the lines

```

if  $k == 1$  then
  | return 1

```

and we replace the line `ret = StrangeAlgo(n, k-2)` by the following code.

```

 $i = 1$ 
while  $i \leq n$  do
  | StrangeAlgo(n, k-1)
  |  $i++$ 
 $ret = k - (k/2) * 2$ 

```

The last line computes 1 if k is odd and 0 otherwise; recall that $/$ is integer division. We need this last line if we want that the output of `StrangeAlgo` is the same as before—note that now we call `StrangeAlgo` with $k-1$ instead of $k-2$.

To see the running time, observe that there is one call with value k for k , and that there are n calls with value $k-1$, n^2 with value $k-2$, n^3 with value $k-3$, $\dots$, and, finally, there are n^k calls with value 0. Since each of the last calls takes $\Omega(1)$ running time, we know that the total running time is at least $n^k \cdot \Omega(1) = \Omega(n^k)$ as claimed.

Advanced Tasks

Challenging tasks for motivated students, but beyond the scope of this course. Their difficulty level is comparable to that of standard computer science courses.

Task 2.8: Cyclic Rotation

Given an array $a[1..n]$ and an integer $k \in [1, n]$, rotate the elements in a by k elements to the left. In other words, at the end when your program stops, for every $1 \leq i \leq n$, $a[i]$ should contain the value that initially was at position $a[(i+k)\%n]$.

Find the following two solutions, both solutions with optimal asymptotic running time and $\Theta(1)$ extra space complexity!

- (a) In the first solution, minimize the number of times you access the array; for this solution, you are allowed to call the function `gcd(i, j)` that returns in time $\Theta(\log^2 i + \log^2 j)$ the greatest common divisor of i and j .

Solution: Time: $\Theta(n)$.

For $i = 1$ up to `gcd(n, k)` (compute the greatest common divisor only once at the beginning), save $a[i]$ in a *temp* variable and move $a[(i+k)\%n]$ to $a[i]$, then $a[(i+2k)\%n]$ to $a[(i+k)\%n]$, then $a[(i+3k)\%n]$ to $a[(i+2k)\%n]$ until it's time to move $a[i]$. Instead of $a[i]$ write the *temp* value on the corresponding position, and continue with the for loop.

The solution is not obvious and you have to use math to prove its correctness.

- (b) For the second solution, the only thing you cannot access the array directly but you are allowed to call the function `Reverse(a, i, j)` that reverses the element order in the subarray `a[i..j]` (in time $\Theta(j - i)$).

Solution: Time: $\Theta(n)$.

The solution is very elegant and simple but I don't want to spoil it to you. I will only tell you as much as this: it's a three-liner of the form:

```
Reverse(a, ?, ?)
Reverse(a, ?, ?)
Reverse(a, ?, ?)
```
